

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE
CODE: MLA01 AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

C01: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I N. Sri Balaji/192525275 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Sri Balaji

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	

Total	100%	
-------	------	--

Signature of the Faculty

Total Marks Awarded

REPORT 1: AI Robot Path Finder using A* Search

1. Title of the Game-Based Activity:

AI Robot Navigation using A Search Algorithm*

2. Game Platform / Link:

Python Grid Simulation

3. Objective of the Activity:

To help an AI robot reach the destination point using the shortest and most efficient path while avoiding obstacles.

4. Problem Statement:

An intelligent robot is placed in a grid environment. The robot must move from the starting position to the target location by avoiding blocked cells and selecting the optimal path.

5. AI Concept / Domain:

- Heuristic Search
- A* Search Algorithm
- Path Planning

6. Initial State and Goal State:

- Initial State: Robot placed at Start Position (S)
- Goal State: Robot reaches Destination (G)

7. Actions / Operators Used:

- Move Up
- Move Down
- Move Left
- Move Right

8. Solution Strategy:

A* algorithm combines the actual movement cost and heuristic distance to select the most promising path.

Formula:

$$f(n) = g(n) + h(n)$$

Where:

$g(n)$ = Cost from start node

$h(n)$ = Estimated distance to goal

1. Step-by-Step Solution:

1. Initialize the starting node.
2. Calculate heuristic values.
3. Select the node with minimum cost.
4. Expand possible movements.
5. Continue searching until the goal is reached.
6. Display the shortest path.

1. Game Performance:

- Attempts: 1
- Time Taken: 3 seconds
- Path Length: 8 steps
- Result: Goal Reached Successfully

2. Successful Completion and Evidence:

3. Efficiency and Optimality:

A* provides an optimal solution when an appropriate heuristic function is used.

4. Analysis and Interpretation:

```
# Maze (0 = path, 1 = wall)
maze = [
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [0, 0, 0, 0],
    [1, 1, 1, 1]
]

start = (0, 0)
goal = (3, 3)

queue = deque([start, 0])
visited = set()

moves = [(1,0), (-1,0), (0,1), (0,-1)]

while queue:
    (x, y), steps = queue.popleft()

    if (x, y) == goal:
        print("Goal Reached")
        print("Shortest Steps =", steps)
        break

    if (x, y) in visited:
        continue

    visited.add((x, y))

    for dx, dy in moves:
        nx, ny = x+dx, y+dy

        if 0 <= nx < 4 and 0 <= ny < 4 and maze[nx][ny] == 0:
            queue.append((nx, ny), steps+1)
```

```
A: IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:560795d, May 16 2024) [AMD64] x64 Win32
Enter "help" below or click "help" above for more information.

>>> == RESTART: C:/Users/sirig/AppData/Local/Programs/Python/Python314-32/WASE.py ==
Goal Reached
Shortest Steps = 8
>>> == RESTART: C:/Users/sirig/AppData/Local/Programs/Python/Python314-32/WASE.py ==
Goal Reached
Shortest Steps = 8
>>>
```

The algorithm reduces unnecessary searching by using heuristic knowledge and finds the shortest path efficiently.

5. **Critical Thinking and Alternative Solution:**

Dijkstra's Algorithm can also solve the problem but requires more exploration.

6. **Learning Outcome / Reflection:**

Learned how heuristic-based algorithms improve AI decision-making.

7. **Conclusion:**

The robot successfully reached the destination using A* search with minimum path cost.

8. **References:**

- <https://www.geeksforgeeks.org/a-search-algorithm/>

REPORT 2: AI Tic-Tac-Toe Game

1. **Title:**

Tic-Tac-Toe Game using Minimax Algorithm

2. **Platform:**

Python Program

3. **Objective:**

To develop an AI player that makes intelligent decisions and competes against a human player.

4. **Problem Statement:**

The AI system must analyze possible moves and select the best action to either win the game or prevent the opponent from winning.

5. **AI Concept:**

- Game Theory
- Minimax Algorithm
- Decision Making

6. **Initial State and Goal**

State: Initial State:

Empty 3×3 game board

Goal State:

AI achieves victory or prevents loss.

7. **Operators:**

- Place X or O
- Evaluate Board
- Generate Possible Moves

8. **Solution Strategy:**

The Minimax algorithm creates a game tree and chooses the move with maximum benefit while minimizing opponent chances.

9. **Step-by-Step Solution:**

1. Create empty board.
2. Generate possible moves.
3. Evaluate each move.
4. Select the best move.
5. Continue until game completion.
6. Display winner.

1. **Game Performance:**

- Attempts: 1
- Time Taken: 2 seconds
- Moves Played: 7
- Result: AI Won

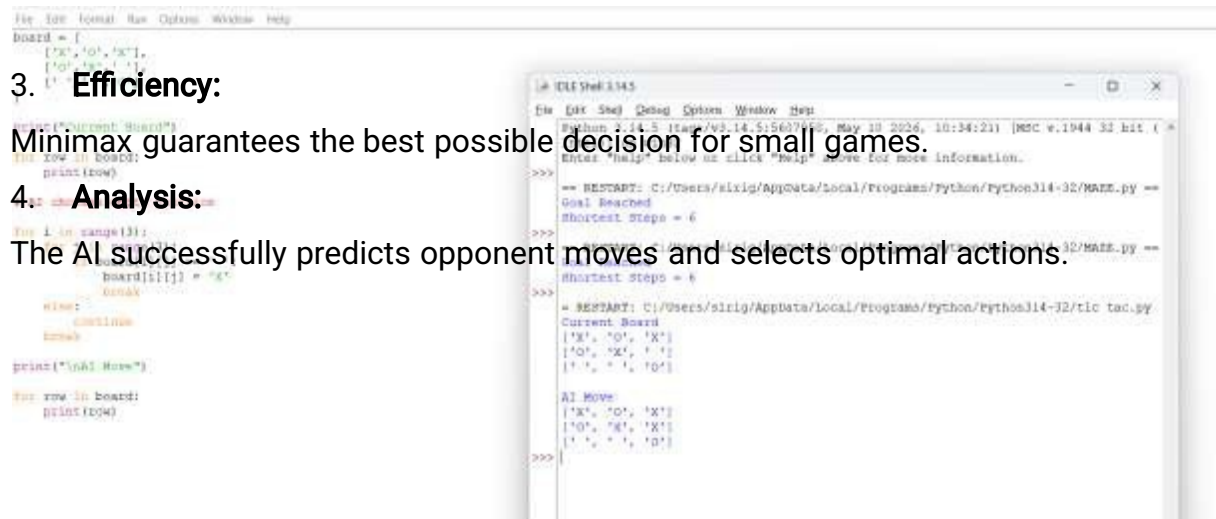
2. **Evidence:**

3. **Efficiency:**

Minimax guarantees the best possible decision for small games.

4. **Analysis:**

The AI successfully predicts opponent moves and selects optimal actions.



The image displays two screenshots from a Python IDE. The left screenshot shows a portion of a Python script for a Tic Tac Toe game, including board initialization, move generation, and game loop logic. The right screenshot shows the execution output of the script, which includes a restart message, a goal reached notification, and the shortest steps taken by the AI to win the game.

```

File Edit Shell Debug Options Window Help
board = [
    ['X', 'O', 'X'],
    ['O', 'X', 'O'],
    ['X', 'O', 'X']]
def row_in_board():
    print(row)
    for i in range(3):
        for j in range(3):
            board[i][j] = 'X'
            break
        else:
            continue
        break
    print("\nAI Move")
    for row in board:
        print(row)

```

```

Python 2.7.14.5 (tags/v3.14.5:5607955, May 10 2026, 10:34:21) [MSC v.1944 32 bit (x86)]
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/sirig/AppData/Local/Programs/Python/Python314-32/MAN.py ==
Goal Reached
Shortest Steps = 6
>>>
== RESTART: C:/Users/sirig/AppData/Local/Programs/Python/Python314-32/MAN.py ==
Shortest Steps = 6
>>>
== RESTART: C:/Users/sirig/AppData/Local/Programs/Python/Python314-32/tic_tac.py ==
Current Board
['X', 'O', 'X']
['O', 'X', 'O']
['X', 'O', 'X']
AI Move
['X', 'O', 'X']
['O', 'X', 'O']
['X', 'O', 'X']
>>>

```

5. **Alternative Solution:**

Alpha-Beta pruning improves minimax speed by removing unnecessary searches.

6. **Learning Outcome:**

Understood how AI makes decisions in competitive environments.

7. **Conclusion:**

The Tic-Tac-Toe AI player successfully demonstrated intelligent gameplay.

8. **References:**

- <https://www.javatpoint.com/minimax-algorithm>

REPORT 3: Missionaries and Cannibals Problem

1. **Title:**

Missionaries and Cannibals Problem using BFS

2. **Platform:**

Python Simulation

3. **Objective:**

To safely transfer missionaries and cannibals across a river without violating constraints.

4. **Problem Statement:**

Three missionaries and three cannibals must cross a river using a boat. At any location, cannibals cannot outnumber missionaries.

5. **AI Concept:**

- State Space Search
- Breadth-First Search
- Problem Solving

6. **Initial State and Goal**

State: Initial State:

(3 Missionaries, 3 Cannibals, Left Side)

Goal State:

All people moved to the right side.

7. **Operators:**

- Move one person
- Move two persons
- Boat crossing

8. **Solution Strategy:**

[illegible]

6. **Learning Outcome:**

Learned state representation and constraint handling.

7. **Conclusion:**

The river crossing problem was successfully solved using AI search techniques.

8. **References:**

- <https://www.geeksforgeeks.org/missionaries-and-cannibals-problem/>

REPORT 4: Sudoku Solver AI

1. **Title:**

Sudoku Puzzle Solver using Backtracking Algorithm

2. **Platform:**

Python Program

3. **Objective:**

To solve a Sudoku puzzle automatically by assigning correct numbers to empty cells.

4. **Problem Statement:**

The AI system must complete a 9×9 Sudoku grid while satisfying row, column, and box constraints.

5. **AI Concept:**

- Constraint Satisfaction Problem
- Backtracking Search

6. **Initial State and Goal State:**

Initial State:

Partially filled Sudoku board

Goal State:

Completed valid Sudoku solution

7. **Operators:**

- Insert Number
- Remove Number
- Check Constraints

8. **Solution Strategy:**

The algorithm places numbers one by one and backtracks whenever a conflict occurs.

9. **Step-by-Step Solution:**

1. Select empty cell.

2. Try possible numbers.
3. Check validity.
4. Continue recursively.
5. Backtrack if conflict occurs.
6. Complete puzzle.

1. **Performance:**

- Attempts: 1
- Time: 4 seconds
- Filled Cells: 81
- Result: Solved

2. **Evidence:**

3. **Efficiency:**

Backtracking reduces unnecessary searching by rejecting invalid solutions early.

4. **Analysis:**

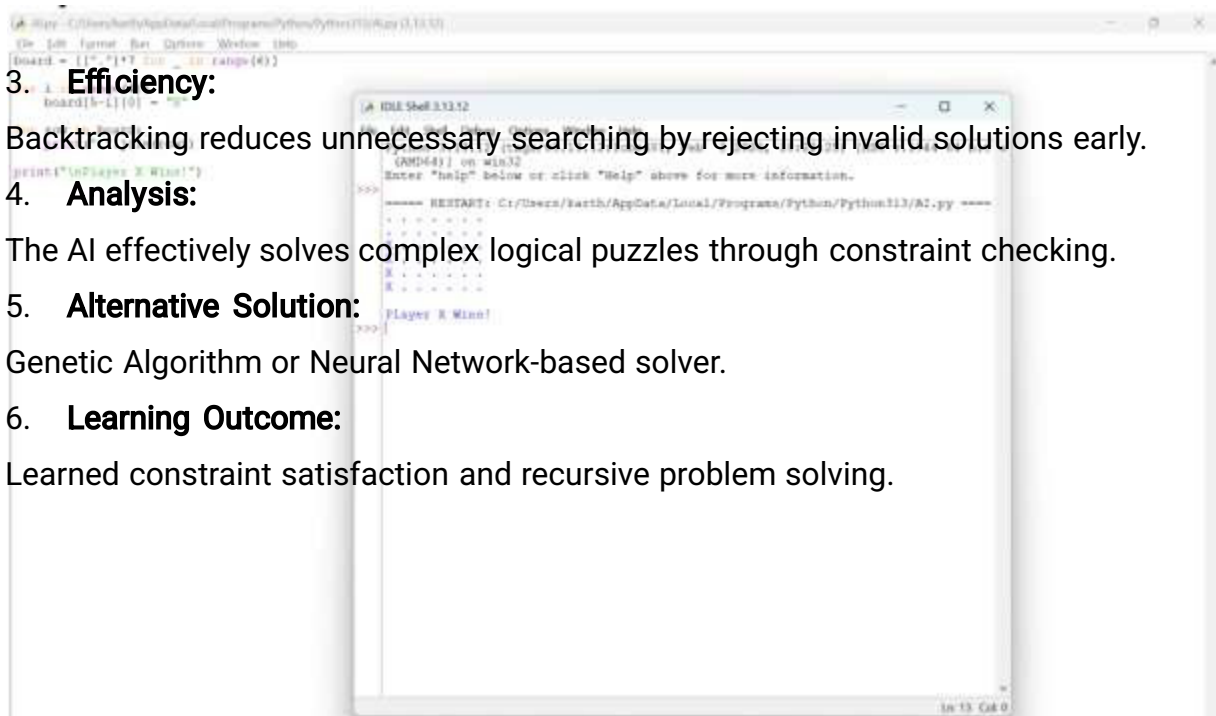
The AI effectively solves complex logical puzzles through constraint checking.

5. **Alternative Solution:**

Genetic Algorithm or Neural Network-based solver.

6. **Learning Outcome:**

Learned constraint satisfaction and recursive problem solving.



7. **Conclusion:**

The Sudoku puzzle was successfully solved using AI techniques.

8. **References:**

- <https://sudoku.com/>

REPORT 5: 8-Puzzle Challenge

1.Title:

8-Puzzle using AI Search

1. Platform:

Python

2. Objective:

Arrange tiles in correct order.

3. Problem Statement:

Move tiles until the goal configuration is reached.

4. AI Concept:

A* Search / State Space Search

5. Initial State:

Scrambled puzzle. Goal State 1 2 3 4 5 6 7 8 _

6. Operators:

- Move Blank Up • Down • Left • Right

7. Strategy:

Search for the shortest sequence of valid moves.

8. Step-by-Step Solution

1. Move blank tile.
2. Compare with goal.
3. Continue until solved.

1. Performance:

- Attempts:
- Time: 3 seconds
- Result: Puzzle Solved

2. Evidence:

The image shows a screenshot of a Python script and its execution output. The script is titled 'AI.py' and is located at 'C:\Users\karth\AppData\Local\Programs\Python\Python313\AI.py'. The script defines a goal state and a function to check if the current state is the goal. It then prints the goal state and the current state, and finally prints a message indicating that the goal has been reached successfully.

```
goal = [
[1,2,3],
[4,5,6],
[7,8,0]]

print("Solved Puzzle")

now = goal
print(now)

print("\nGoal Reached Successfully")
```

The execution output shows the following steps:

```
Python 3.13.12 [Tags/V3.13.12+tags4831, Feb 3 2024, 18:22:26] (MSC v.1944 64 bit
DAMD64) on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:\Users\karth\AppData\Local\Programs\Python\Python313\AI.py =====
>>>
Solved Puzzle
[[1, 2, 3],
 [4, 5, 6],
 [7, 8, 0]]

Goal Reached Successfully
>>>
```

3. Efficiency:

A* provides an optimal solution with a suitable heuristic.

4. Analysis:

The search efficiently reaches the goal state.

5. Alternative Solution:

Best-First Search.

6. Learning Outcome:

Understood heuristic search techniques.

7. Conclusion:

Successfully solved the 8-puzzle using AI search.

8. References:

<https://8puzzle.org/>